

Scott Dunphy – section 2

Intro to AI

PA1

9/23/26

Written Analysis

Question 1:

Precise formula:

Let n be the current state and g be the goal.

$D(n)$ = the set of positions where n differs from the goal

$K = \text{length}(D(n))$, the number of such positions.

If $k = 0$, $h(n) = 0$ for all, this is the goal state

Steps:

1. Formula: $h(n) = k$
2. Cost it Lower Bounds: The unavoidable remaining cost it lower bounds is the number of actions still needed. Each position that isn't the same as the goal will need at least one action of its own to reach the goal.
3. Why it can't overestimate: Every path from state n to g has at least k actions. Every replacement costs 1, so every remaining path costs atleast k , as well as the optimal path: $h^*(n) \geq k = h(n)$. example: ask $\rightarrow$ why, the $h(\text{why}) = 3$, but $h^*=5$, because some letters at certain positions have to change more than once to stay on real words, which gives $\text{delta} = -2$.

Scrabble:

1. Formula: $h(n) = \sum_i^{D(n)} v(g[i])$, where v is the value of the letter according to the scrabble table.
2. Cost it lower bounds: the cost of inserting the goal letter at each wrong position. Scrabble cost charges based on the letter being inserted.
3. Why it can't overestimate: every mismatched position has to receive its goal letter eventually, so any solution pays atleast the sum of this. Extra letter changes to match words in the dictionary only add more to the cost, so $h^* \geq h(n)$
Example: beck $\rightarrow$ duck = $2 + 1 = 3$

Frequency:

1. $h(n) = (1 + r(g)) + (k-1) * (1 + r_{\min})$ if $k \geq 1$, otherwise $h(n) = 0$ if $k = 0$
where $r(g)$ is the rarity of the goal word, and $r_{\min}$ is the minimum over all words.
2. what it lower bounds: there are atleast $k-1$ more actions, and each costs atleast $1 + r_{\min}$. also, every solution ends by the goal word, which costs $1 + r(g)$
3. why it cant be overestimated: the last move always costs $1 + r(g)$, and every other required move costs at least as much as landing on the most common word in the dictionary, so no real path can be cheaper than this estimate.
4. Example: beck -> duck, $k = 2$
Last step = $1 + 7.230 = 8.230$
Other step at cheapest possible: $1 * 1.016 = 1.016$
 $H(n) = 8.230 + 1.016 = 9.246$
 $H^*(n) = 16.264$

Question 2:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.86	808	38.86	634	21.53%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.24	1084	38.24	301	72.23%

- Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy:
yes, both returned the same optimal cost for each one. This is because both are optimal because all three heuristics are admissible.
- For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?
For WHY->ASK: Scrabble with 98.30%, and for BEAR->DUCK: Scrabble with 99.80%.
- For each word pair, under which cost measure was the heuristic least informative?
Frequency for both pairs, 21.53% for WHY->ASK and 72.23% for BEAR->DUCK.
- Explain the differences using the definitions of your heuristics, not runtime alone.
Steps and scrabble both count exactly the letters that still need to change. Its why they do so well on BEAR->DUCK because each letter changes straight to the goal letters, and having A* be a low expansion makes the percentage better. Frequency is a bad measure because it assumes every in-between word is as cheap as the minimum, but in reality that is not the case. The closer h is to the real cost, the more nodes A* skips.

Question 3:

Steps

1. Yes
2. Each action costs 1, given a path w_1 to w_2 with n actions, both paths from w_1 to w_2 and w_2 to w_1 must equal n .

Scrabble

1. No
2. A move costs the value of the letter inserted. changing the letter B to A will insert the value of A, while going from A to B will insert the value of B, so they are different and not reversible
3. CAT $\rightarrow$ HAT insert h = 4
HAT $\rightarrow$ CAT inserts c = 3

Frequency

1. No
2. a move costs 1 + rarity of the word you land on. Going A $\rightarrow$ B pays for B and going B $\rightarrow$ A pays for A. it charges every word except the start, the reversed path charges every word except the end.
3. CAT rarity = 6.887, Hat rarity = 6.446
CAT $\rightarrow$ HAT = 1 + r(hat) = 7.446
HAT $\rightarrow$ CAT = 1 + r(cat) = 7.887

Question 4:

1. Gemini 3.1 pro

Prompt: In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

$$h_1(n) = \begin{cases} 0 & \text{if } n = g \\ h_0(n) + \text{rarity}(g) & \text{if } n \neq g \end{cases}$$

Proposed heuristic:

2. Full formula:

Define the proposed heuristic $h_1(n)$ as:

$$h_1(n) = \begin{cases} 0 & \text{if } n = g \\ h_0(n) + \text{rarity}(g) & \text{if } n \neq g \end{cases}$$

Where:

- n is the current word state.
- g is the goal word state.
- $h_0(n)$ is the number of mismatched letters between n and g .
- $\text{rarity}(g)$ is the dictionary rarity value of the goal word.

3. Yes it is admissible:

Let $n \neq g$, and take any legal path $n = w_0 \rightarrow w_1 \rightarrow \dots \rightarrow w_m = g$.

Each action changes one letter, and $k = h_0(n)$ letters are wrong, so $m \geq k$.

The last action enters g and costs $1 + \text{rarity}(g)$.

Each of the other $m - 1$ actions enters some word w with $\text{rarity}(w) \geq 0$, so each costs at least 1.

Total cost $\geq (m - 1) + 1 + \text{rarity}(g) = m + \text{rarity}(g) \geq k + \text{rarity}(g) = h_1(n)$.

4. No repairs needed because its not invalid
5. Each run used `astar_search(DC(initial, goal, "frequency"))`

Problem	Heuristic	Solution cost	Nodes expanded	vs. h_0
WHY → ASK	h_0	38.86	755	(baseline)
WHY → ASK	h_1 (Gemini)	38.86	634	-16.0%
WHY → ASK	h_2 (= my dc.py)	38.86	634	-16.0%
BEAR → DUCK	h_0	38.24	720	(baseline)
BEAR → DUCK	h_1 (Gemini)	38.24	302	-58.1%
BEAR → DUCK	h_2 (= my dc.py)	38.24	301	-58.2%

Conclusion: all three found the same optimal cost and path. H_1 expanded 16% fewer nodes than h_0 , which supports using it.

AI usage: All code and written responses were done by me. The only used during this project was for clarification on certain ideas, such as understanding delta.

Claude Opus 5.5:

Prompt: can you explain the deltas and what they mean after running dctest.py

It then explained how the deltas associated with the program are just a way to see the heuristic in action, where a negative is how far and a positive is a failure or inadmissible.

It was only used for clarification on ideas like that.